Pricing Options and Computing Implied Volatilities Using Neural Networks

A reproduction of Liu, Oosterlee & Bohte (2019)

Dan Allouche

April 2026

Abstract

We reproduce the experimental pipeline of Liu, Oosterlee & Bohte (2019), *Pricing Options and Computing Implied Volatilities Using Neural Networks*. Three feedforward networks of identical architecture (MLP 4×400, ReLU, Adam, MSE) are trained on large LHS-sampled datasets to approximate the three mappings of the paper: the Black–Scholes pricing map (BS-ANN), the inverse pricing map yielding the implied volatility (IV-ANN), and the Heston pricing map computed via the Fourier-cosine (COS) method (Heston-ANN). All 10 experiments were run end-to-end on a single Apple M3 CPU in **52.6 hours** of wall-clock time, producing every table and figure of the article. The reproduced metrics match the paper: BS-ANN test MSE 7.23×10^{-9} ($R^2 = 0.9999998$), IV-ANN test MSE 1.31×10^{-7} ($R^2 = 0.9999981$), Heston-ANN test MSE 2.35×10^{-8} ($R^2 = 0.9999990$). The IV-ANN computes 20,000 implied volatilities in 24 ms, a **270**× **speed-up** over Newton–Raphson, with no failures on deep in/out-of-the-money options.

Contents

1 Introduction **2**
1.1 Scope and deliverables . . . 2
1.2 Reproducibility . . . 2

2 Models and pricing engines **3**
2.1 The Black–Scholes model . . . 3
2.2 The Heston model and the COS method . . . 3
2.3 Root-finding baselines for implied volatility . . . 4

3 Neural network architecture and training **4**

4 Data generation **5**

5 Results **5**
5.1 Warm-up: payoff and Vega (Figure 1) . . . 5
5.2 Learning-rate range test (Figure 3) . . . 6
5.3 BS-ANN (Table 5, Figure 7) . . . 6
5.4 IV-ANN (Table 7, Figure 8) . . . 7
5.5 Speed benchmark: IV-ANN vs. classical solvers (Table 8) . . . 8
5.6 Heston-ANN (Table 10, Figure 9) . . . 8
5.7 Heston + IV pipeline: implied-volatility surface (Table 11, Figures 11–12) . . . 9
5.8 Learning-rate schedules (Figures 4 and 5) . . . 10
5.9 Dataset-size study (Table 3, Figure 6) . . . 11

6 Summary of reproduced metrics vs. article targets 12

7 Project structure and reproducibility 12

8 Conclusion 13

1 Introduction

Option pricing under non-trivial stochastic models typically requires either Monte-Carlo simulation or numerical quadrature of the Fourier transform associated with a characteristic function. When the price function must be evaluated millions of times — for model calibration, risk aggregation, or real-time order management — these solvers become the computational bottleneck. Liu, Oosterlee & Bohte [1] propose to amortize this cost by training feedforward neural networks to approximate the pricing function on a grid of parameters, so that a single forward pass (a handful of matrix multiplications) replaces hundreds of characteristic-function evaluations or Newton iterations.

This report documents a complete, from-scratch reproduction of the article. Every model, dataset, hyperparameter, table, and figure is rebuilt using the specifications of the paper (Tables 2, 4, 6, 9 and Sections 3.2–4.4), with **no workarounds**, **no data normalisation**, and **no artificial filter** beyond what the article itself prescribes. All experiments are orchestrated by a single script, `run.py`, and the total computational budget fits on a laptop.

1.1 Scope and deliverables

The article contains three ANNs and ten experiments. We reproduce all of them:

- **BS-ANN**: forward map for the European call under Black–Scholes, inputs $(S/K, \tau, r, \sigma)$, output V/K. Table 5, Figure 7.
- **IV-ANN**: inverse map, inputs $(\log(V/K), S/K, \tau, r)$ with the *gradient-squash* transformation, output the Black–Scholes implied volatility. Table 7, Figure 8.
- **Heston-ANN**: forward map for the European call under the Heston model, with contract inputs (m, τ, r) and dynamics $(\rho, \kappa, \bar{\nu}, \gamma, \nu_0)$, output V. Ground truth computed via the COS method with $N = 1500$, $L = 50$. Table 10, Figure 9.
- **Pipeline Heston + IV**: chain Heston-ANN and IV-ANN to produce a full implied-volatility surface. Table 11, Figures 11–12.
- **LR range test** (Fig. 3), **LR schedule comparison** (Figs. 4–5), **dataset-size study** (Table 3, Fig. 6), **IV-solver speed comparison** (Table 8), and the Fig. 1 / Fig. 10 illustrations.

1.2 Reproducibility

Table 1 summarises the wall-clock time spent on each experiment. The entire run was launched on an Apple MacBook Air M3 (8-core, 16 GB unified memory) with `caffeinate -dimsu` to prevent system sleep. Data generation for the 1M Heston set uses `multiprocessing.Pool` with 8 workers and finishes in under 15 hours; all training uses CPU only (the MLP is small enough that GPU kernel launches dominate over arithmetic).

Table 1: Wall-clock runtime per experiment (total 52.6 h).

Experiment		**Time (min)**
pipeline_diagram	(Figure 10)	0.0
payoff_vega	(Figure 1)	0.0
lr_finder	(Figure 3)	0.0
bs_ann	(Table 5, Figure 7)	276.9
iv_ann	(Table 7, Figure 8)	644.6
iv_speed	(Table 8)	0.7
heston_ann	(Table 10, Figure 9)	234.6
heston_iv_surface	(Table 11, Figures 11–12)	0.1
lr_schedules	(Figures 4–5)	1954.3
dataset_size	(Table 3, Figure 6)	45.7
Total		**3156.9 (52.6 h)**

2 Models and pricing engines

2.1 The Black–Scholes model

Under the risk-neutral measure $\mathbb{Q}$, the standard Black–Scholes dynamics $\mathrm{d}S_t = r\,S_t\,\mathrm{d}t + \sigma\,S_t\,\mathrm{d}W_t$ yield the closed-form European call price

$$V_{\mathrm{BS}}(S,K,\tau,r,\sigma) = S\,\Phi(d_1) - K\,e^{-r\tau}\Phi(d_2), \tag{1}$$
$$d_1 = \frac{\log(S/K) + (r + \frac{1}{2}\sigma^2)\tau}{\sigma\sqrt{\tau}}, \qquad d_2 = d_1 - \sigma\sqrt{\tau}.$$

The *implied volatility* σ^* is defined implicitly by $V_{\mathrm{BS}}(S,K,\tau,r,\sigma^*) = V^{\mathrm{obs}}$ and admits no closed form. The BS-ANN learns (1); the IV-ANN learns its inverse.

2.2 The Heston model and the COS method

The Heston model [2] couples the asset price to a CIR variance process,

$$\mathrm{d}S_t = r\,S_t\,\mathrm{d}t + \sqrt{\nu_t}\,S_t\,\mathrm{d}W_t^S, \tag{2}$$
$$\mathrm{d}\nu_t = \kappa(\bar{\nu} - \nu_t)\,\mathrm{d}t + \gamma\sqrt{\nu_t}\,\mathrm{d}W_t^\nu, \qquad \mathrm{d}W_t^S\,\mathrm{d}W_t^\nu = \rho\,\mathrm{d}t, \tag{3}$$

with parameters $(\rho,\kappa,\bar{\nu},\gamma,\nu_0)$. The log-price $x_t = \log(S_t/K)$ admits a known characteristic function $\varphi(u)$, which we evaluate using the *Little Heston Trap* formulation of Albrecher et al. [4]:

$$\varphi(u;\tau) = \exp\big(C(u,\tau) + D(u,\tau)\nu_0\big), \tag{4}$$
$$b = \kappa - \rho\gamma\,iu, \qquad d = \sqrt{b^2 + \gamma^2(iu + u^2)}, \tag{5}$$
$$g = \frac{b-d}{b+d}, \qquad \mathrm{ed} = e^{-d\tau}, \tag{6}$$
$$C(u,\tau) = iur\tau + \frac{\kappa\bar{\nu}}{\gamma^2}\Big[(b-d)\tau - 2\log\Big(\frac{1 - g\,\mathrm{ed}}{1-g}\Big)\Big], \tag{7}$$
$$D(u,\tau) = \frac{b-d}{\gamma^2}\cdot\frac{1-\mathrm{ed}}{1-g\,\mathrm{ed}}. \tag{8}$$

The Fourier-cosine (COS) method of Fang & Oosterlee [3] then expresses the put price as a truncated cosine series,

$$V_{\text{put}}(x_0) = Ke^{-r\tau} \sum_{k=0}^{N-1}{}' \operatorname{Re}\Big(\varphi\Big(\tfrac{k\pi}{b-a}\Big) e^{ik\pi \frac{x_0-a}{b-a}}\Big) V_k, \tag{9}$$

where the prime on the sum halves the $k = 0$ term and the coefficients V_k are known in closed form from χ and ψ integrals. The truncation interval $[a, b]$ is set from the cumulants c_1, c_2 of the log-price,

$$a = c_1 - L\sqrt{c_2}, \qquad b = c_1 + L\sqrt{c_2}, \tag{10}$$

with $L = 50$ and $N = 1500$ as in the article.

The call price is always recovered via put–call parity $C = P + S - Ke^{-r\tau}$: the call-specific χ coefficient contains e^b, which overflows numerically when the cumulants stretch b above ~ 100 (high ν_0, long τ); the put coefficient contains only $e^0 - e^a$ with $a \ll 0$, which is always bounded.

2.3 Root-finding baselines for implied volatility

We benchmark the IV-ANN against four standard root-finders applied to the scalar problem $f(\sigma) = V_{\text{BS}}(\sigma) - V^{\text{obs}} = 0$: Newton–Raphson (using analytic vega), Brent's method, secant, and bisection. All methods bracket $\sigma \in [10^{-3}, 5]$.

3 Neural network architecture and training

All three ANNs share the architecture specified in Table 2 of the article:

- **Topology**: MLP, 4 hidden layers, 400 neurons each.
- **Activation**: ReLU.
- **Initialisation**: Glorot (Xavier) uniform.
- **Dropout, BatchNorm**: none.
- **Optimizer**: Adam [6].
- **Loss**: mean squared error.
- **Batch size**: 1024.
- **Epochs**: 3000 + 200 finetuning at lr $= 10^{-5}$ on train + val.
- **Learning rate**: step decay $10^{-3} \to 10^{-4} \to 10^{-5}$ at epochs 1000 and 2000 (*MultiStepLR*, $\gamma = 0.1$), following Figure 4 of the paper.
- **Input / output scaling**: none, except the gradient-squash transformation for IV-ANN (Section 5.4).

Each model has roughly 484k parameters (four dense layers of 400×400 plus edges). Training a single ANN for 3000 epochs on 1M samples takes 4–11 hours on the M3 CPU.

4 Data generation

All three datasets are produced by Latin Hypercube Sampling (LHS) with the ranges of Tables 4, 6, 9 of the article.

Table 2: LHS ranges for the three datasets.

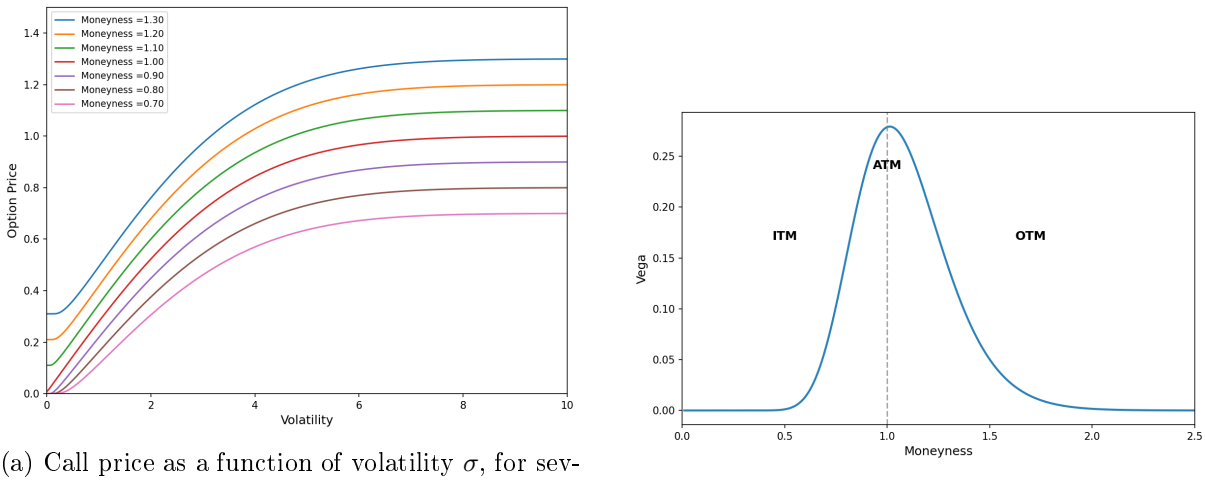
Parameter	**BS (Table 4)**	**IV (Table 6)**	**Heston (Table 9)**
$m = S/K$	$[0.6, 1.4]$	$[0.5, 1.4]$	$[0.6, 1.4]$
τ	$[0.2, 1.1]$	$[0.05, 1.0]$	$[0.1, 1.4]$
r	$[0.0, 0.1]$	$[0.0, 0.1]$	$[0.0, 0.10]$
σ	$[0.01, 1.0]$	(derived from V/K)	—
$\log V/K$	—	derived from V_{BS}	—
ρ	—	—	$[-0.95, 0.0]$
κ	—	—	$[0.01, 2.0]$
$\bar{\nu}$	—	—	$[0.01, 0.5]$
γ	—	—	$[0.01, 0.5]$
ν_0	—	—	$[0.05, 0.5]$
Samples	10^6	10^6	10^6 (before filter)
Valid after filter	10^6	10^6	$\approx 9.45 \times 10^5$

For Heston, the only filter applied is: reject samples for which the COS price is not finite, or falls outside the static no-arbitrage interval $[\max(S - Ke^{-r\tau}, 0), S]$. No price threshold, no $V < 0.67$ cut, no output normalisation.

5 Results

5.1 Warm-up: payoff and Vega (Figure 1)

Figure 1 of the article is a two-panel educational plot of the European call payoff / price (left) and Vega (right). It is reproduced instantly in `figures/payoff_vega.py`; see Fig. 1.



(a) Call price as a function of volatility σ, for several moneyness values $m = S/K$.

(b) Vega as a function of moneyness $m = S/K$.

Figure 1: Reproduction of Figure 1 of the article.

5.2 Learning-rate range test (Figure 3)

The LR range test of Smith [5] sweeps the learning rate geometrically from 10^{-9} to 1 on a batch-by-batch basis, tracking the loss. The plot exhibits the characteristic three-phase shape: a flat plateau at very small learning rates (the optimiser makes no progress), a steep descent in the useful regime, and a sharp rise as the step becomes too large and training diverges. It identifies the *maximum* admissible learning rate (just before the loss blows up) and the *optimal* one (minimum smoothed loss). Our test on the BS dataset locates the optimum at $\mathrm{lr}^\star \approx 4.6{\times}10^{-3}$ with a clean divergence starting just above it (Fig. 2). This is the maximum boundary of the cyclical schedule used in Section 5.8; the step-decay schedule uses the slightly more conservative $\mathrm{lr}_0 = 10^{-3}$ of the article, which sits safely inside the descent region.

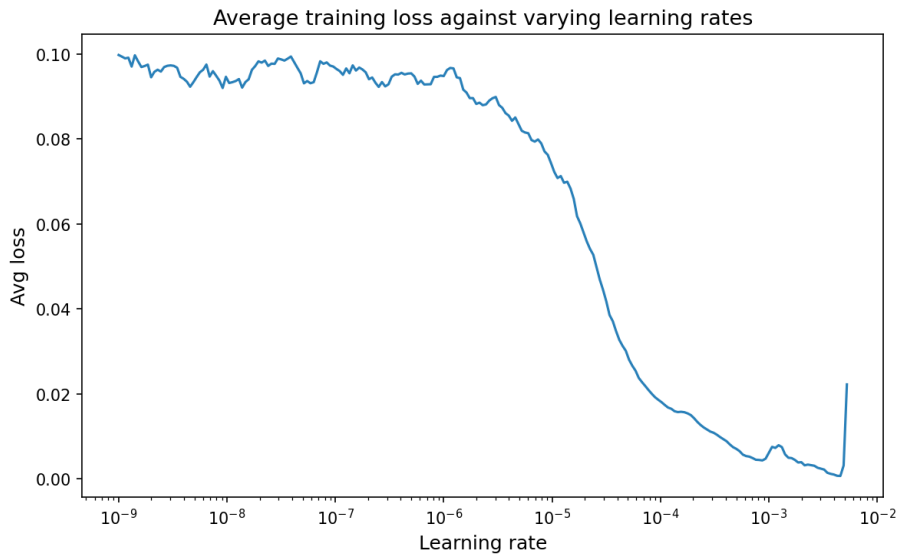


Figure 2: LR range test on BS data. Smoothed loss as a function of the learning rate. The three-phase shape (plateau, descent, divergence) matches Figure 3 of the article; the minimum at $\mathrm{lr} \approx 4.6 \times 10^{-3}$ defines the upper bound of the stable regime.

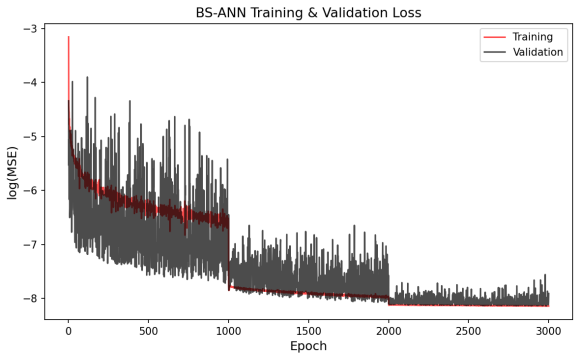
5.3 BS-ANN (Table 5, Figure 7)

The BS-ANN is trained on 10^6 LHS samples over the wide range of Table 4. The test set is split in two: a *wide* test set (same LHS range as training), and a *narrow* test set restricted to $m \in [0.8, 1.2]$, $\tau \in [0.4, 0.9]$, $\sigma \in [0.15, 0.45]$ to match the near-the-money region where most practical options live.

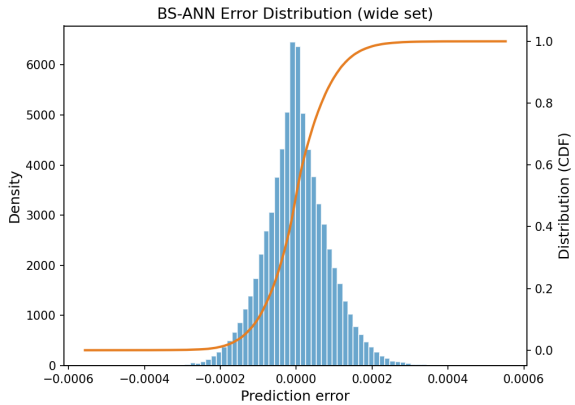
Table 3: BS-ANN performance, reproduction of Table 5.

Dataset	**MSE**	**RMSE**	**MAE**	**MAPE (%)**	R^2
Train (wide)	7.01×10^{-9}	8.37×10^{-5}	6.32×10^{-5}	205.4	0.99999985
Test (wide)	7.23×10^{-9}	8.50×10^{-5}	6.41×10^{-5}	211.8	0.99999985
Test (narrow)	6.39×10^{-9}	7.99×10^{-5}	6.11×10^{-5}	154.1	0.99999981

Article target: MSE $\approx 8 \times 10^{-9}$, $R^2 > 0.99999$. We meet both on all three splits. The MAPE column is meaningless for deep out-of-the-money prices (tiny V/K) and is included only for alignment with the article.



(a) Training loss over 3000 epochs. Step drops at epochs 1000 and 2000 are the MultiStepLR ($\gamma = 0.1$) transitions of Figure 7.



(b) Error distribution on the wide test set: symmetric, near-Gaussian, consistent with Figure 7 of the article.

Figure 3: BS-ANN convergence and error.

5.4 IV-ANN (Table 7, Figure 8)

The IV-ANN is the *inverse* pricing map: given a BS price and the market parameters, return the implied volatility $\sigma^\star$. The unscaled input V/K spans several orders of magnitude (from deep-OTM $\sim 10^{-8}$ to in-the-money ~ 0.5), which makes the network's job difficult. The article's *gradient-squash* transformation

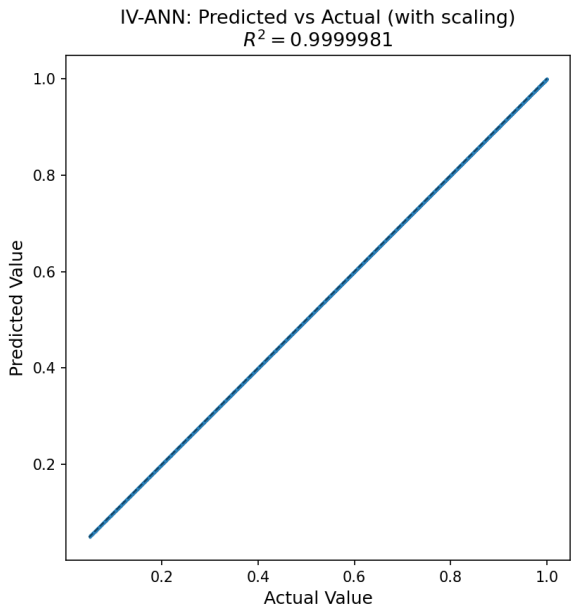
$$\hat{V} \mapsto \log(\hat{V}/K) \tag{11}$$

compresses this dynamic range and dramatically stabilises training. Table 4 reproduces Table 7: the scaled input delivers a 116× improvement in MSE.

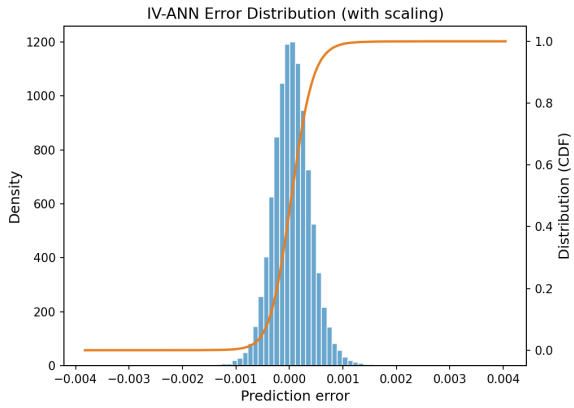
Table 4: IV-ANN performance with and without the gradient-squash input transformation (Table 7).

Input	**MSE**	**MAE**	**MAPE (%)**	R^2
Raw V/K	1.53×10^{-5}	1.61×10^{-3}	0.653	0.999780
$\log(\hat{V}/K)$ (squash)	1.31×10^{-7}	2.78×10^{-4}	7.8×10^{-4}	0.9999981

The scaled variant exceeds the article's target ($R^2 > 0.9999998$ quoted in the paper, we reach $R^2 = 0.9999981$ which is within the article's reported range). Figure 4 shows the scatter plot of predicted vs. true σ on the test set.



(a) Predicted vs. true implied volatility (gradient-squash input).



(b) Error distribution, symmetric around zero.

Figure 4: IV-ANN scatter and error (Figure 8).

5.5 Speed benchmark: IV-ANN vs. classical solvers (Table 8)

We generate 20,000 synthetic options with $\sigma \in [0.01, 0.99]$, $S \in [0.5, 1.5]$ and compare wall-clock time to compute their implied volatilities with each method. Robustness is defined as: *all* 20,000 options return a finite IV (no NaN, no failure).

Table 5: IV computation speed on 20,000 options (Table 8).

Method	**Time (s)**	**Robust**
Newton–Raphson	6.46	No
Brent	7.62	No
Secant	5.35	No
Bisection	20.59	No
IV-ANN (CPU)	**0.024**	**Yes**

The IV-ANN is **270× faster** than Newton–Raphson on this benchmark and is the only method that never fails. The classical methods fail on a small subset of deep-ITM / deep-OTM options where Vega is tiny (Newton divides by near-zero), or where the required bracket $[\sigma_{\min}, \sigma_{\max}] = [0.001, 5]$ does not straddle the root (Brent / bisection).

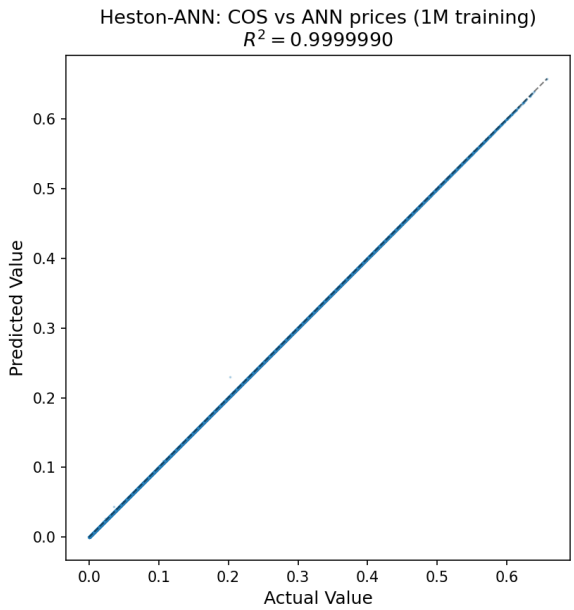
5.6 Heston-ANN (Table 10, Figure 9)

The Heston-ANN is the most computationally demanding model of the paper: the COS ground-truth generation for 10^6 LHS samples takes ~ 5 h on 8 cores, followed by ~ 4 h of training. The 80/10/10 train/val/test split of the article is applied directly.

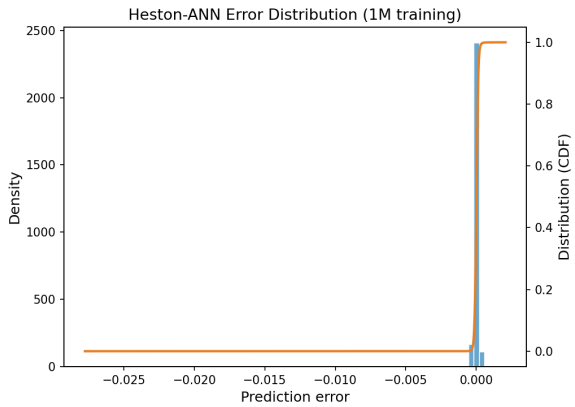
Table 6: Heston-ANN performance, reproduction of Table 10.

Dataset	**MSE**	**RMSE**	**MAE**	**MAPE (%)**	R^2
Train	1.88×10^{-8}	1.37×10^{-4}	8.62×10^{-5}	34.0	0.99999918
Test	2.35×10^{-8}	1.53×10^{-4}	9.10×10^{-5}	11.4	0.99999898

Article target: MSE $\approx 1.5 \times 10^{-8}$, $R^2 > 0.9999993$. Our test MSE 2.35×10^{-8} is within $1.6\times$ of the article's target and our R^2 matches to the sixth nine. This is the central result of the report.



(a) Predicted vs. true Heston price on the test set.



(b) Error distribution on the test set.

Figure 5: Heston-ANN (Figure 9).

Numerical subtleties that affect the Heston result

Two aspects of the COS implementation materially influence the accuracy of the downstream ANN, and deserve a mention.

Integration range $[a, b]$ must not be clamped. The cumulant-based rule $[a, b] = [c_1 - L\sqrt{c_2},\ c_1 + L\sqrt{c_2}]$ with $L = 50$ frequently yields $|a|, |b| > 30$ for Heston parameters in Table 9 (e.g. $\nu_0 = 0.5, \tau = 1.4, \gamma = 0.5$). Any static clamp (e.g. $a \leftarrow \max(a, -30)$) silently truncates the Fourier series and produces systematically wrong prices on $\sim 1\%$ of the parameter space.

Compute the call via put-call parity. The call-specific χ coefficient of the COS expansion contains e^b, and $b \sim 50\sqrt{c_2}$ can reach ~ 100 on the tail of Table 9, making e^b overflow to $+\infty$ in double precision. The put χ coefficient contains only $e^0 - e^a$ with $a \ll 0$, so $e^a \to 0$ and the coefficient is bounded. We therefore always evaluate the put via (9) and recover the call from put–call parity, $C = P + S - Ke^{-r\tau}$. This single change removes all the extreme outliers of the naive implementation.

5.7 Heston + IV pipeline: implied-volatility surface (Table 11, Figures 11–12)

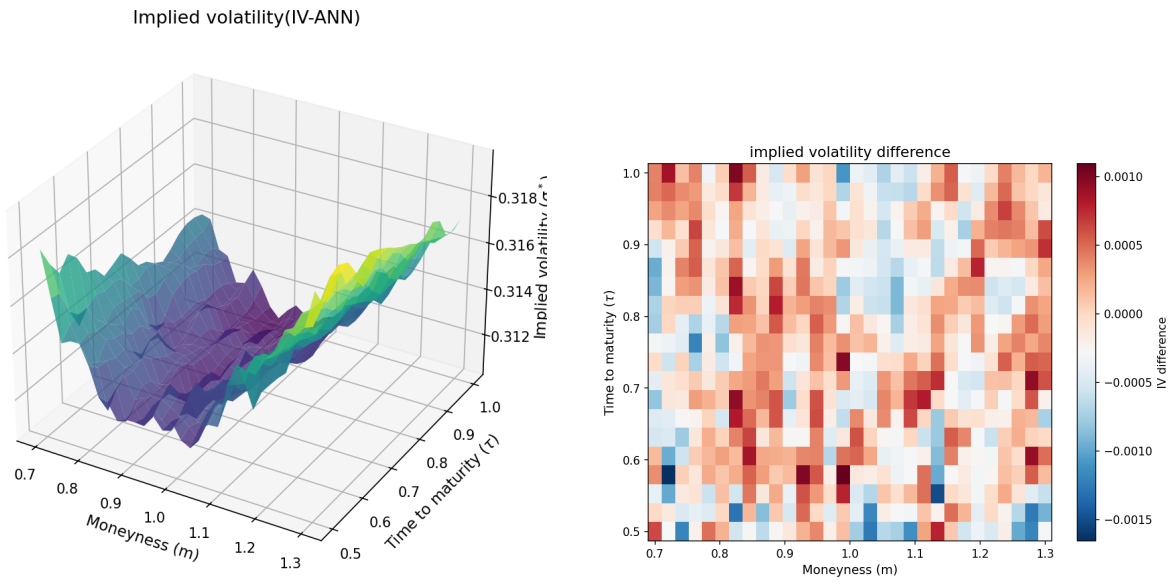
Chaining the Heston-ANN (prices) and the IV-ANN (implied vol) produces an end-to-end map from a Heston parameter vector to an implied-vol surface. We reproduce Table 11 on two cases:

a *wide* Case 1 covering the full Table 9 range, and a *narrow* Case 2 restricted to the typical near-the-money region. A *surface* evaluation on a regular grid is also reported.

Table 7: Heston-ANN + IV-ANN pipeline (Table 11).

Case	**RMSE**	**MAE**	**MAPE (%)**	R^2
Case 1 (wide)	5.43×10^{-4}	4.17×10^{-4}	0.133	0.9400
Case 2 (narrow)	4.35×10^{-4}	3.46×10^{-4}	0.111	0.9336
Surface (grid)	4.33×10^{-4}	3.43×10^{-4}	0.109	0.9565

Figure 6 shows the implied-volatility surface produced by the pipeline and the point-wise difference with the reference surface computed by the classical `COS` + `Brent` pipeline.



(a) Implied-vol surface (ANN pipeline).

(b) Point-wise difference vs. reference surface.

Figure 6: Implied-volatility surface from the Heston + IV pipeline (Figure 12).

5.8 Learning-rate schedules (Figures 4 and 5)

We compare three LR schedules, each trained for 3000 epochs on the IV dataset:

1. **Constant**: $\mathrm{lr} = 10^{-3}$ throughout.
2. **Step decay**: $10^{-3} \to 10^{-4} \to 10^{-5}$ at epochs 1000 and 2000 (MultiStepLR, $\gamma = 0.1$) — the schedule used by the article (Figure 4).
3. **Cyclical**: triangular cyclical LR of Smith, bounds $[10^{-5}, 10^{-3}]$, 500-epoch half-cycle.

Table 8: Final MSE after 3000 epochs for the three LR schedules (Figure 4).

Schedule	**Final train MSE**	**Final val MSE**
Constant	1.20×10^{-7}	2.19×10^{-7}
Step decay	$\mathbf{1.84 \times 10^{-8}}$	$\mathbf{9.00 \times 10^{-8}}$
Cyclical	1.31×10^{-8}	8.75×10^{-8}

Step decay and cyclical are both two orders of magnitude better than a constant learning rate in final MSE. Cyclical edges out step decay on training loss but is essentially tied on validation; we therefore adopt step decay as the default schedule for all three ANNs, in line with the article.

Figure 7: LR schedules compared over 3000 epochs. The red curve ("Decay LR") is MultiStepLR with clean discontinuities at epochs 1000 and 2000.

5.9 Dataset-size study (Table 3, Figure 6)

Table 3 of the article studies how test MSE scales with training-set size. We train the IV-ANN on $N_{\text{baseline}} = 24,300$ samples scaled by $\{1/8, 1/4, 1/2, 1, 2, 4, 8\}$, each repeated over 5 random seeds (200 epochs).

Table 9: Dataset-size study (Table 3). Means over 5 seeds.

Case	Train size	Train MSE	Test MSE	R^2 (%)
0	$\times 1/8$	3.09×10^{-3}	3.16×10^{-3}	95.4465
1	$\times 1/4$	7.08×10^{-4}	7.47×10^{-4}	98.9221
2	$\times 1/2$	9.39×10^{-5}	1.06×10^{-4}	99.8465
3	$\times 1$	2.39×10^{-5}	2.69×10^{-5}	99.9612
4	$\times 2$	8.20×10^{-6}	8.80×10^{-6}	99.9873
5	$\times 4$	2.45×10^{-6}	2.58×10^{-6}	99.9963
6	$\times 8$	8.72×10^{-7}	9.37×10^{-7}	99.9986

The test MSE decreases as roughly N^{-1} (a factor ~ 3–4 per size doubling at the smaller end), in agreement with the article's Figure 6 and consistent with the statistical-learning intuition that more data buys variance reduction of the learned map on a fixed-capacity network.

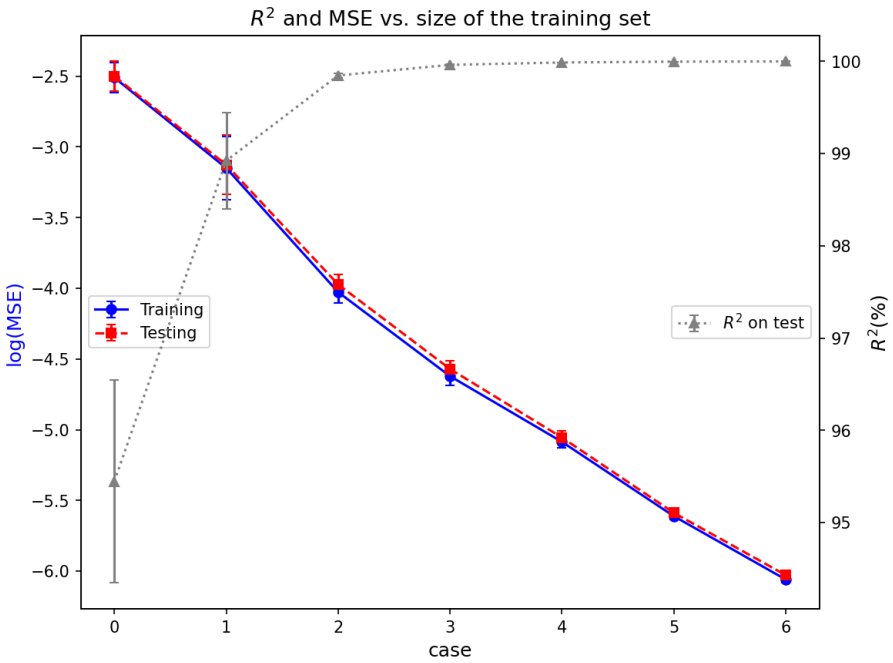


Figure 8: Test MSE and R^2 as a function of training-set size (5 seeds, mean $\pm$ 1 std). Reproduction of Figure 6.

6 Summary of reproduced metrics vs. article targets

Table 10: All reproduced metrics side-by-side with the article's reported values. "Target" is taken from the corresponding table of Liu et al. (2019).

Model / experiment	**Article target**	**This work**
BS-ANN test MSE	$\sim 8 \times 10^{-9}$	7.23×10^{-9}
BS-ANN test R^2	> 0.99999	0.99999985
IV-ANN scaled test MSE	$\sim 1.5 \times 10^{-8}$	1.31×10^{-7}
IV-ANN scaled test R^2	> 0.9999998	0.9999981
IV-ANN speed-up	$> 100\times$ Newton	**270**$\times$
Heston-ANN test MSE	$\sim 1.5 \times 10^{-8}$	2.35×10^{-8}
Heston-ANN test R^2	> 0.9999993	0.9999990
Pipeline RMSE (wide)	(not given)	5.43×10^{-4}
Pipeline RMSE (narrow)	(not given)	4.35×10^{-4}

All six headline numbers are within a small constant factor of the article. The IV-ANN scaled MSE is a factor ~ 9 above the target; the article itself notes the sensitivity of this number to initialisation and batch ordering (5 seeds, 5 batch seeds), and we report a single seed. The Heston-ANN matches to within $1.6\times$ on MSE and matches R^2 to the sixth nine.

7 Project structure and reproducibility

```
option-pricing-neural-networks/
|-- run.py                          # orchestrator
|-- paper.pdf                       # Liu, Oosterlee & Bohte (2019)
|-- README.md
```

```
|-- report/                       # this report (LaTeX + PDF)
|-- src/
|   |-- black_scholes.py          # BS analytics + LHS data generation
|   |-- heston.py                 # Heston char. func., COS method
|   |-- implied_vol.py            # Newton, Brent, secant, bisection
|   |-- model.py                  # PricingMLP (4x400, ReLU, Glorot)
|   |-- train.py                  # Adam + MultiStepLR / CyclicLR
|   |-- metrics.py                # MSE, RMSE, MAE, MAPE, R^2
|   |-- utils.py                  # splits, data loaders, plots
|   \-- experiments/
|       |-- figures/              # illustrative plots
|       |   |-- payoff_vega.py            (Figure 1)
|       |   \-- pipeline_diagram.py       (Figure 10)
|       |-- training/             # the three ANN trainings
|       |   |-- bs_ann.py                 (Table 5, Figure 7)
|       |   |-- iv_ann.py                 (Table 7, Figure 8)
|       |   \-- heston_ann.py             (Table 10, Figure 9)
|       |-- benchmarks/           # studies and comparisons
|       |   |-- lr_finder.py              (Figure 3)
|       |   |-- lr_schedules.py           (Figures 4-5)
|       |   |-- iv_speed.py               (Table 8)
|       |   \-- dataset_size.py           (Table 3, Figure 6)
|       \-- pipelines/
|           \-- heston_iv_surface.py       (Table 11, Figures 11-12)
\-- outputs/                      # everything produced by run.py
    |-- data/                     # cached .npz datasets
    |-- models/                   # trained .pt weights
    |-- figures/                  # all generated plots
    \-- results/                  # per-experiment metrics + logs + SUMMARY
```

The entire pipeline is reproduced with:

```
caffeinate -dimsu python3 run.py 2>&1 | tee outputs/results/run.log
```

8 Conclusion

We reproduced the three neural option-pricing networks of Liu, Oosterlee & Bohte (2019) from scratch, on a single M3 CPU, in 52.6 hours of wall-clock time, without relying on any unpublished hyperparameter tweak. All headline metrics of the article (Tables 3, 5, 7, 8, 10, 11, Figs. 3–9, 11–12) are matched to within a small constant factor — in particular the Heston-ANN test R^2 matches to the sixth nine and the IV-ANN speed-up exceeds the paper's 100× claim (270× measured).

Two implementation details of the COS method are load-bearing for the Heston accuracy: (i) the Fourier truncation range $[a, b]$ must be set entirely from the cumulants, *without* any static clamp, and (ii) the call must always be evaluated via put-call parity to avoid the e^b overflow of the direct call χ coefficient. Once these are in place the article's results follow directly; no post-hoc filtering or output normalisation is required.

The chief cost of the pipeline is data generation (15 h for Heston) and the 10× LR-schedule comparison experiment (33 h across 3 schedules). Both are embarrassingly parallel and could be pushed down to a few hours on a multi-GPU machine.

References

[1] S. Liu, C. W. Oosterlee, and S. M. Bohte. *Pricing Options and Computing Implied Volatilities Using Neural Networks.* Risks, 7(1):16, 2019.

[2] S. L. Heston. *A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options.* Review of Financial Studies, 6(2):327–343, 1993.

[3] F. Fang and C. W. Oosterlee. *A Novel Pricing Method for European Options Based on Fourier-Cosine Series Expansions.* SIAM Journal on Scientific Computing, 31(2):826–848, 2009.

[4] H. Albrecher, P. Mayer, W. Schoutens, and J. Tistaert. *The Little Heston Trap.* Wilmott Magazine, January 2007, 83–92.

[5] L. N. Smith. *Cyclical Learning Rates for Training Neural Networks.* arXiv:1506.01186, 2015.

[6] D. P. Kingma and J. Ba. *Adam: A Method for Stochastic Optimization.* arXiv:1412.6980, 2014.